



PROJETO 01 – DESENVOLVIMENTO WEB III

Desenvolvimento de Software Multiplataforma

Gabriela Cardoso dos Santos
Thalita Mirela Leao de Macedo

São Paulo
2006

1. Identificação dos Integrantes

- Gabriela Cardoso dos Santos
- Thalita Mirela Leao de Macedo

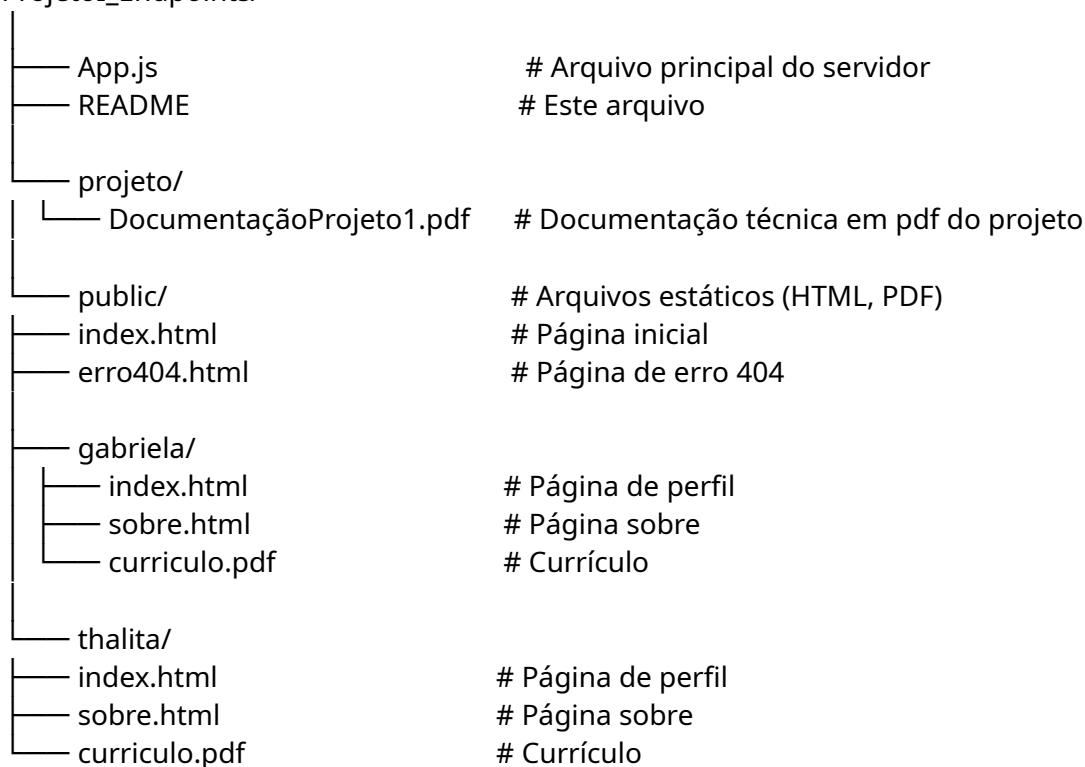
2. Objetivo do Projeto

O objetivo deste projeto é desenvolver uma aplicação web utilizando Node.js para apresentar o perfil dos dois integrantes da dupla e demonstrar na prática os conceitos fundamentais do backend, incluindo a criação de um servidor HTTP, definição de rotas (endpoints), leitura de arquivos do sistema e navegação entre páginas web e documentos em PDF.

3. Estrutura das Rotas Desenvolvidas e do projeto

Rota	Método	Content-Type	Arquivo Servido	Status
/	GET	text/html	public/index.html	200
/gabriela	GET	text/html	public/gabriela/index.html	200
/gabriela/sobre	GET	text/html	public/gabriela/sobre.html	200
/gabriela/curriculo	GET	application/pdf	public/gabriela/curriculo.pdf	200
/thalita	GET	text/html	public/thalita/index.html	200
/thalita/sobre	GET	text/html	public/thalita/sobre.html	200
/thalita/curriculo	GET	application/pdf	public/thalita/curriculo.pdf	200
projeto	GET	application/pdf	projeto/DocumentaçãoProjeto1.pdf	200
/* (outros)	GET	text/html	public/erro404.html	404

ProjetoI_Endpoints/



4. Explicação do Funcionamento da Aplicação

Fluxo de Funcionamento:

1. **Inicialização:** Node.js carrega os módulos necessários (http, url, fs)
2. **Criação do Servidor:** http.createServer() com callback de requisição
3. **Recebimento de Requisição:** Para cada requisição, o servidor:
 - Faz parse da URL usando url.parse()
 - Verifica qual rota foi chamada com if/else if
 - Define o Content-Type apropriado com response.writeHead()
 - Chama readFile() para ler o arquivo assincronamente
4. **Leitura Assíncrona:** fs.readFile() lê o arquivo
5. **Resposta:** Envia o conteúdo ao cliente com response.end(data)
6. **Escuta:** Servidor fica escutando na *porta 3000*

5. Todos os Códigos-Fonte Utilizados no Projeto

Módulos Nativos do Node.js:

```
// Carregar os módulos:  
const http = require('http'); //Trata protocolo http  
const url = require('url'); // Usado para url (end-points)  
const fs = require('fs'); // Fille-system (acessar arquivos)
```

Função Principal – readFile():

```
//Função para ler arquivo e enviar no response http:  
function readFile(response, file) {  
    // Fazer a leitura do arquivo de forma assincrona:  
    fs.readFile(file, function (err, data) {  
        // Após ler, escrever no response (http) o conteudo:  
        response.end(data)  
    });  
}
```

Callback de Requisição:

```
// Aplicação isolada - com callback:
let callback = function (request, response) {

  // Faz o Parse da URL, separa os endpoints:
  let parts = url.parse(request.url);

  //Verificar os endpoints:
  if (parts.path == "/" ) {
    response.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    readFile(response, "public/index.html");
  }

  // Endpoints para o perfil da Gabriela:
  else if (parts.path == "/gabriela") {
    response.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    readFile(response, "public/gabriela/index.html");
  }
  else if (parts.path == "/gabriela/sobre") {
    response.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    readFile(response, "public/gabriela/sobre.html");
  }
  else if (parts.path == "/gabriela/curriculo") {
    response.writeHead(200, { "Content-Type": "application/pdf" });
    readFile(response, "public/gabriela/curriculo.pdf");
  }

  // Endpoints para o perfil da Thalita:
  else if (parts.path == "/thalita") {
    response.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    readFile(response, "public/thalita/index.html");
  }
  else if (parts.path == "/thalita/sobre") {
    response.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    readFile(response, "public/thalita/sobre.html");
  }
  else if (parts.path == "/thalita/curriculo") {
    response.writeHead(200, { "Content-Type": "application/pdf" });
    readFile(response, "public/thalita/curriculo.pdf");
  }

  // Projeto documentado no Pdf na rota:
  else if (parts.path == "/projeto") {
    response.writeHead(200, { 'content-type': 'application/pdf' });
    readFile(response, "projeto/DocumentaçãoProjeto1.pdf");
  }

  // Erro 404 - Página não encontrada:
  else {
    response.writeHead(404, { "Content-Type": "text/html; charset=utf-8" });
    readFile(response, "public/erro404.html");
  }
};
```

Criação e Inicialização:

```
// Criar o servidor http:  
let server = http.createServer(callback);  
  
// Configurar a porta de escuta do servidor:  
server.listen(3000);  
console.log("Servidor rodando em http://localhost:3000");
```

6. Explicação dos Principais Trechos de Código

Com o objetivo de evidenciar o funcionamento interno do nosso servidor, descrevemos abaixo a arquitetura e os principais blocos de código presentes no arquivo principal da aplicação.

Módulos Nativos Incorporados:

- **http**: Módulo base responsável por instanciar o servidor web e gerenciar todo o fluxo de requisições e respostas sob o protocolo HTTP.
- **url**: Empregado para fazer o *parse* (análise) minucioso dos endereços acessados, desmembrando a URL para estruturar corretamente o sistema de rotas.
- **fs (File System)**: Encarregado de interagir com o sistema de arquivos do servidor, permitindo a leitura e manipulação dos documentos físicos do projeto, como as páginas web e os PDFs.

Processamento de Arquivos (readFile): Essa função atua realizando a leitura não-bloqueante (assíncrona) dos arquivos armazenados em disco. Assim que o documento (seja um **.pdf** ou **.html**) é carregado, ela se encarrega de devolver esses dados brutos diretamente no corpo da resposta HTTP.

Mecanismo de Roteamento: Sempre que uma requisição é feita, a string da URL é avaliada e separada em blocos lógicos. Esse processo nos permite identificar com precisão qual *endpoint* o usuário deseja acessar, gravando esse destino em uma variável (como `parts.path`) para direcionar o fluxo da aplicação.

Estrutura da Rota Principal (/):

- O caminho `/` aponta para o diretório raiz, carregando a página inicial do projeto.
- Quando acionada, ela devolve um status **200 (OK)**, confirmando o sucesso da requisição.
- O cabeçalho HTTP é ajustado para `text/html`, instruindo o navegador a interpretar o retorno como uma estrutura de página web.
- A leitura do documento físico que compõe a interface é feita de imediato pela função de leitura do sistema.

Disponibilização de Documentos PDF: Para as rotas que entregam os currículos ou a documentação, o fluxo sofre uma pequena alteração. O cabeçalho Content-Type é modificado para application/pdf. Esse detalhe é obrigatório, pois sinaliza ao navegador do usuário que ele deve acionar seu leitor de PDF embutido, evitando que o arquivo seja exibido como um texto desconfigurado.

Captura de Exceções (Erro 404): Implementamos uma validação para garantir que acessos a caminhos não mapeados no sistema sejam tratados. Se o endereço digitado não coincidir com as rotas definidas, o código aciona um bloco condicional (else) de exceção. Como resultado, o servidor emite o código **404 (Not Found)** e renderiza uma página amigável e personalizada de "página não encontrada".

7. Justificativa das Decisões Adotadas

Decisão	Justificativa
Servidor Nativo (sem framework)	Demonstrar compreensão dos fundamentos HTTP e Node.js puro
Callbacks assíncronos	Não bloquear thread principal; melhor performance
Parse manual de URLs	Entender roteamento sem abstrações de framework
Verificação com if/else	Simples e direto para número pequeno de rotas
Content-Type dinâmico	Servir corretamente diferentes tipos (HTML, PDF, etc)
Charset UTF-8	Suporte a caracteres especiais portugueses/acentos
Estrutura em public/	Separação clara entre código e recursos estáticos

8. Considerações Finais sobre o Projeto

O desenvolvimento desta aplicação cumpriu com êxito o objetivo de aplicar, na prática, os conceitos fundamentais de arquitetura cliente-servidor utilizando Node.js. Ao optarmos por construir a infraestrutura de rede e o roteamento utilizando exclusivamente os módulos nativos da plataforma (http, fs e url), sem a abstração de *frameworks* de terceiros, o projeto proporcionou uma visão profunda e detalhada sobre o funcionamento do protocolo HTTP "por baixo dos panos".

Durante a execução, ficou evidente a importância do gerenciamento correto de cabeçalhos (*headers*), como o controle estrito do Content-Type para diferenciar o envio de páginas web (text/html) da renderização de documentos estáticos (application/pdf). Além disso, a manipulação assíncrona do sistema de arquivos demonstrou ser uma prática essencial para garantir que o servidor não sofra gargalos de processamento ao atender a múltiplas requisições simultâneas.

Do ponto de vista funcional, o sistema atende plenamente aos requisitos propostos, servindo de maneira ágil e segura os portfólios, currículos (das integrantes Gabriela e Thalita) e a documentação oficial do projeto. A implementação da rota de exceção (Erro 404) reforça a preocupação da equipe com a experiência do usuário e com a resiliência da aplicação diante de requisições inesperadas.

Em suma, a construção manual desse sistema de rotas consolida uma base técnica robusta. O domínio dessas operações de baixo nível é um passo indispensável na nossa jornada como estudantes de desenvolvimento de software, capacitando a equipe para lidar com arquiteturas mais complexas, APIs avançadas e integrações futuras com maior segurança e propriedade técnica.